SOFTWARE MODELLING

AMIRA AJANAKU (25111699) / **SENZO LUKHELE** (24691497) / **JAY MACASKILL** (25198387)

08 SEPTEMBER 2026 / COS214 PRACTICAL 4 (TASK FORGE)

Task 1

Task Forge is a work management system for software development projects, modelling work breakdown structures where Projects, Tasks and Epics are treated uniformly and move through a lifecycle. The system allows for flexible traversal of the work hierarchy and addition of responsibilities such as priority and logging.

We have elected to do a software delivery hierarchy. This system acts as a means to track work being done and the lifecycle of Tasks/Projects/Epics.

- The Composite pattern is used such that a WorkGroup and a WorkItem can be treated uniformly through a common WorkComponent interface.
- A WorkItem could represent something such as a Task whilst a WorkGroup could represent a Project or an Epic, etc. However, all WorkComponents have a lifecycle which ranges from Todo →InProgress →Blocked →Done. This serves as the State design pattern within the system.
- Further, we make use of a TaskDecorator to decorate a task further by priority with the PriorityDecorator, or we can make use of a LoggingDecorator as well – this serves as the Decorator design pattern in the system.
- The Iterator design pattern allows us to traverse through a Composite. There is a DepthFirstIterator which will iterate through the whole Composite tree as well as an ActiveOnlyIterator which is filtered to traverse through only active tasks.
- These patterns interact meaningfully to model a realistic system. Tasks/Projects/Epics do not exist in the real world independent of a lifecycle, priority level, logs or a means to go through all Tasks/Projects/Epics and a means to filter which of these are active and need tending to.
    - The patterns therefore do not act independently, rather they interleave and assist with increasing the meaningfulness of the design.

Notably on memory management:

- WorkGroup owns its children as a vector of pointers and the destructor deletes the children. This is a WorkItem within a WorkGroup will still need to be referred to until the WorkGroup is complete, the Composite tree is hence deleted cascadingly.
- Additionally, Decorator does not own its wrapped item. As mentioned, a WorkComponent is either standalone as a WorkItem, in which case the caller deletes it, or it's contained within a WorkGroup, in which case the WorkGroup deletes it.

- If the Decorator deleted its wrapped item, this may lead to double freeing resulting in memory-related issues.

Pattern	Participants	
Iterator	Iterator	WorkIterator
	ConcreteIterator	DepthFirstIterator, ActiveOnlyIterator
	Aggregate	WorkComponent
	ConcreteAggregate	WorkGroup, WorkItem
Composite	Component	WorkComponent
	Leaf	WorkItem
	Composite	WorkGroup
Decorator	Component	WorkComponent
	ConcreteComponent	WorkItem
	Decorator	TaskDecorator
	ConcreteDecorator	PriorityDecorator, LoggingDecorator
State	State	TaskState
	ConcreteState	TodoState, InProgressState, BlockedState, DoneState
	Context	WorkItem

Task 5

//Debugging (Segfault)

While running the interactive demo (demo.cpp), selecting option 7 ("Everything At Once") caused the program to crash with a segmentation fault immediately after the "Composite Removal" section printed:

⚠ This WorkComponent could not be found in this WorkGroup!
✅ WorkComponent added to WorkGroup successfully!
📊 PROJECT STATUS 📊
Segmentation fault (core dumped)

Debugging process (GDB):

The program was rebuilt with debug symbols (-g) and run inside GDB to catch the crash at the exact point of failure:

```
gdb ./demo
(gdb) run
```

After reproducing the crash, a backtrace was taken:

```
(gdb) bt
#0  0x000055555555b9d8 in WorkGroup::appendTo (this=0x555555578ad0, out=...) at
    WorkGroup.cpp:161
#1  0x000055555555e68a in status () at demo.cpp:192
#2  0x000055555555ee65 in main () at demo.cpp:286
```

This identified the crash as occurring inside WorkGroup::appendTo, called from status(). Inspecting the source at that line:

```
(gdb) list WorkGroup.cpp:161
159         out.push_back(this);
160         for (size_t i = 0; i < children.size(); i ++)
161             children[i]->appendTo(out);
```

Inspecting the children vector at the point of the crash revealed the actual problem - one element was a null pointer:

```
(gdb) print children[2]
$4 = (WorkComponent *) 0x0
```

Cause

Tracing back to removingComposite() in demo.cpp:

cpp

```
WorkComponent* removed = project->remove(task);
project->add(removed);
```

WorkGroup::remove() only searches the direct children of the group it is called on. Since task ("UI Design") is nested two levels deep
(project -> frontend -> UI Design), calling project->remove(task) correctly fails to find it and returns nullptr:

```
cpp
cout << " ⚠ This WorkComponent could not be found in this WorkGroup!\n";
return nullptr;
```

However, the calling code in demo.cpp never checked whether remove() succeeded before immediately re-adding the result:

```
cpp
project->add(removed); // removed is nullptr here
```

This inserted a literal null pointer into project's children vector. The crash did not occur immediately — it only surfaced later, when status()
called appendTo(), which iterates over every child and calls appendTo() on each one without checking for null, dereferencing the null pointer and
crashing.

Correction

The fix is to check the return value of remove() before using it:

```
cpp
WorkComponent* removed = project->remove(task);
if (removed != nullptr)
{
   project->add(removed);
}
else
{
   cout << "Could not move the task - it wasn't a direct child of this group.\n";
}
```

This prevents a null pointer from ever entering the composite structure. A more complete fix would also be to reconsider WorkGroup::remove()'s scope it currently only searches one level, so a design decision is needed on whether removal should search only direct children (and calling code must know the correct group) or recurse through the whole subtree.